Engineered-Feature MLP for Spray Penetration (v2: Hard Physics Prior on Pressure and Diameter)

Mie Spray Postprocessing Project

2026 年 4 月 10 日

Message

Claim: the sparse-feature instability shown for the v1 direct-feature MLP can be removed at its source by replacing the pressure/diameter inputs with a single hard physical scaling group taken from the Hiroyasu–Arai / Zhou penetration laws, and by training the network on the resulting non-dimensional target.

What this deck does:

- recap the v1 sparse-feature failure mode
- state the physical scaling group used in v2
- describe exactly which features are removed, which are added, and how the target is reparameterized
- describe the new training scaffolding (group splits, pretrain collapse check, two variants)

Recap: What v1 Showed

v1 used 9 direct features:

$$[\text{time}, \theta_{\rm tilt}, n_{\rm plumes}, d_n, t_i, \log P_{\rm inj}, \log P_{\rm ch}, \log \Delta P, P_{\rm cb}]$$

Observed pathology (see `slides_sparse_feature_instability`):

- nozzle diameter has only 6 levels and is not uniformly available across pressures
- dense MLP sweeps over d_n produce 5–6 gradient sign changes and $p95\,|d^2\mu/dd_n^2| \sim 10^7$
- injection pressure shows worst-secant-deviation ratio > 1 on weakly supported slices

Diagnosis: the network is allowed to invent continuous interpolation between sparse levels because the data does not constrain $\partial\mu/\partial d_n$ or $\partial\mu/\partial \log P_{\rm inj}$ almost anywhere.

Engineering Move: Use the Theory Directly

The Hiroyasu–Arai post-breakup branch and the Zhou et al. EOI extension share one well-supported scaling group. Reading the post-breakup branch

$$S(t) \;\propto\; K_p \left(\frac{\Delta P}{\rho_a}\right)^{1/4} d_n^{1/2}\, t^{1/2},$$

the entire dependence on injection pressure, chamber pressure, chamber density and nozzle diameter collapses to one scalar

$$\boxed{A \;=\; \left(\frac{\Delta P}{\rho_a}\right)^{1/4} d_n^{1/2}}$$

which has units of length and acts as a per-condition length scale.

Idea: stop asking the network to learn d_n, $\log P_{\mathrm{inj}}$, $\log P_{\mathrm{ch}}$, $\log \Delta P$ as four separate sparse inputs. Use A as a hard divisor on the target and feed the network the non-dimensional trajectory.

Computing A from the Test Matrix

Per curve, A is determined entirely from the operating point:

- d_n from `nozzle_properties.diameter_mm` in the test matrix JSON
- $\Delta P = P_{\mathrm{inj}} - P_{\mathrm{ch}}$ from the per-row pressures
- ρ_a from the linear law $\rho_a = 5\,\mathrm{kg/m^3} \times P_{\mathrm{ch}}/4.42\,$bar, calibrated against the explicit `chamber_density_kg_per_m3` entries provided for Nozzle1–8

Implementation reference:

- `engineered_feature_common.build_canonical_feature_table` computes `ambient_pressure_bar_phys`, `ambient_density_kg_m3`, `delta_pressure_bar_phys`, `A_scale`, `log_A`
- the canonicalization is the same for DS300 (derived) and Nozzle1–8 (looked up), so all datasets share one consistent A

New Feature Set

Removed (4 sparse direct features):

$$\{\, d_n,\ \log P_{\mathrm{inj}},\ \log P_{\mathrm{ch}},\ \log \Delta P \,\}$$

Two variants are exposed in `FEATURE_COLUMNS_BY_VARIANT`:

- `a_only` (5 features):

  $$[\mathsf{time}_{\mathrm{norm}},\ \theta_{\mathrm{tilt},z},\ n_{\mathrm{plumes},z},\ t_{i,z},\ P_{\mathrm{cb},z}]$$

  A never enters as an input; it only acts as a divisor on the target.
- `a_plus_log_a` (6 features):

  $$[\mathsf{time}_{\mathrm{norm}},\ \theta_{\mathrm{tilt},z},\ n_{\mathrm{plumes},z},\ t_{i,z},\ P_{\mathrm{cb},z},\ \log A_z]$$

  A also appears once as a one-dimensional residual axis.

Why two variants: `a_only` is the strongest commitment to the Zhou scaling. `a_plus_log_a` lets the network express residual dependence on the scaling group as a single normalized axis instead of four sparse axes.

Target Reparameterization

v1: the network predicted $\mu(t)$ and $\log \sigma^2(t)$ on the physical penetration $S(t)$ in mm.

v2: the network predicts the same two heads on the non-dimensional target

$$\hat{S}(t) = S(t)/A, \qquad \hat{\sigma}^2(t) = \sigma^2(t)/A^2.$$

At inference, the physical mean and standard deviation are recovered exactly by

$$\mu_S(t) = A\,\hat{\mu}(t), \qquad \sigma_S(t) = A\,\hat{\sigma}(t).$$

Implementation reference: `PenetrationDataset` stores `target_scaled = penetration / A_scale` per row. The Stage-1 MSE and Stage-2 NLL objectives in `engineered_feature_common` consume `target_scaled` directly, while `toy_inference_from_run` multiplies the predicted mean and std by `A_scale` when reporting in millimetres.

Why This Should Help the Sparse-Feature Pathology

Mechanism: the four sparse axes that produced spurious curvature in v1 are no longer free inputs. Their entire effect is now expressed through one analytic factor A whose exponents are fixed by theory rather than being learned from a 6-level grid.

Expected consequences:

- $\partial \mu_S / \partial d_n$ inherits the closed-form $d_n^{1/2}$ shape; the network has no remaining degrees of freedom to invent oscillations along d_n
- $\partial \mu_S / \partial P_{\mathrm{inj}}$, $\partial \mu_S / \partial P_{\mathrm{ch}}$ inherit the closed-form $(\Delta P / \rho_a)^{1/4}$ shape, with positive sensitivity by construction
- all remaining learning capacity is spent on the dense, well-supported axes (time, tilt, plumes, injection duration, control backpressure)

Honest caveat: this only helps if the Zhou scaling actually collapses the trajectories on the present data. That is why a pretrain collapse check is now part of the training driver.

Pretrain Collapse Check

Before any optimization, `run_pretrain_collapse_check` reconstructs the representative trajectories from the four fitted constants, divides each by its own A, and tests whether

$$\hat{S}(t) \;\approx\; \text{function of the remaining (dense) features only.}$$

What it produces:

- per-time-slice variance of $\hat{S}(t)$ versus the original $S(t)$
- a linear regression R^2 of $\hat{S}(t)$ on the kept features
- a binary `passed` flag, surfaced in the run directory

Role in the pipeline: if the precheck fails, training aborts unless `--allow-failed-precheck` is passed. This makes the physical assumption *falsifiable* on the actual dataset rather than asserted.

Other Pipeline Changes

Group-aware splits. `assign_splits_by_group` assigns whole `sample_group_id` groups to train/val/test instead of permuting individual rows. This prevents leakage between curves that share an operating point.

Stage-1 / Stage-2 driver split. `train_stage1_mse.py` and `train_stage2_nll.py` replace the v1 monolithic notebooks. Stage 2 warm-starts from a Stage-1 engineered run directory and reuses its scaler state, so the variant choice and the A-scaling are guaranteed identical between stages.

Loss objectives. `compute_loss_stage1_engineered` and `compute_loss_stage2_engineered_nll` keep the v1 monotonicity and post-transient concavity priors, but apply them on $\hat{\mu}(t)$ instead of $\mu(t)$. The shape constraints transfer cleanly because dividing by a positive constant A preserves sign and curvature.

Inference path. `toy_inference_from_run.py` now branches on `infer_feature_family`: legacy v1 runs follow the 9-feature path, engineered v2 runs canonicalize the raw input through the same ρ_a, ΔP, A pipeline used in training and rescale the model outputs by A before plotting.

Engineering Takeaway

What v2 commits to:

- the Hiroyasu–Arai post-breakup scaling group $A = (\Delta P/\rho_a)^{1/4} d_n^{1/2}$ is treated as a hard prior, not a soft regularizer
- the network is no longer asked to learn a continuous law over a 6-point diameter grid or a 4-point pressure grid
- the resulting model is falsifiable: if the collapse check fails, the prior is wrong on this dataset and training stops

What still has to be evaluated post-training:

- rerun the v1 sparse-feature instability diagnostics (`compare_sparse_feature_stability`) and confirm that the secant-deviation ratios and curvature norms drop materially
- verify that the held-out NLL on physical S is at least as good as v1 once the Jacobian factor $\log A$ is accounted for
- compare `a_only` versus `a_plus_log_a` on diagnostics, not just on validation loss